

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Лабораторна робота №2

Частина 4

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Мережа ART-1/ART-2. Класифікація новинних статей за тематикою (спорт, економіка, політика)”

Варіант №3

Виконав:

ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Левкович М.В.

Мета роботи

Засвоїти принципи функціонування нейронних мереж ART-1/ART-2 та набути практичних навичок їх застосування для класифікації текстових даних

Індивідуальне завдання

Виконати кластеризацію неперервних векторів (текстових, сенсорних або сигналів) за допомогою мережі ART-2, дослідити вплив параметрів на якість класифікації.

1. Підготовка даних
 - Обрати відкритий набір даних (новини, телеметрія, аудіо, сенсори).
 - Описати поля, виконати очищення та нормалізацію ознак (Z-score або [0,1]).
 - Для текстів – сформувати TF-IDF або ембеддинги.
2. Формування векторів
 - Представити об'єкти у вигляді векторів дійсних чисел.
 - Нормалізувати всі вектори до довжини 1.
3. Навчання мережі ART-2
 - Реалізувати або використати готову модель.
 - Початкові параметри: $\rho=0.87$, $\theta=0.08$, $\eta=0.6$, $\alpha=1.0$.
 - Провести 2–3 епохи навчання, варіюючи ρ та θ .
4. Оцінювання
 - Зіставити отримані кластери з реальними категоріями.
 - Метрики: Purity, ARI, NMI, кількість кластерів.
5. Візуалізація
 - Побудувати 2D-проєкцію (PCA/t-SNE) кольорами кластерів.
 - Побудувати графік Purity або NMI залежно від ρ .
6. Висновки
 - Порівняти якість кластеризації при різних параметрах.
 - Пояснити, як змінюється кількість кластерів і стабільність мережі.

Типові налаштування: $\rho=0.8-0.95$, $\theta=0.05-0.1$, $\eta=0.4-0.7$, epochs=2.
Метрики: Purity, NMI (тексти); ARI, Silhouette (сенсорні дані).

№	Тематика / Дані	Тип ознак	Початкові параметри ART-2	Індивідуальне завдання студента	Метрики оцінювання
3	Статті: IT / медицина / право	S-BERT ембеддинги	$\rho = 0.85$, $\theta = 0.0$, $\eta = 0.5$	Порівняти результати TF-IDF та S-BERT	NMI, Silhouette

№	Тематика / Дані	Типові ознаки	Рекомендований датасет	Джерело / посилання
3	Статті: IT / медицина / право	S-BERT ембеддинги	arXiv Dataset	Kaggle – arXiv Metadata

Хід роботи

1. Імпорт бібліотек та завантаження даних

На цьому етапі я підключив усі необхідні бібліотеки для обробки текстів, побудови векторних представлень та подальшої кластеризації. Я встановив sentence-transformers для отримання S-BERT ембеддингів, а також sklearn, umap-learn, matplotlib і seaborn для візуалізації та оцінки результатів.

Далі я завантажив відкритий набір даних arXiv Metadata із Kaggle, у якому зберігається інформація про наукові статті (назва, анотація, категорії). Для цього я використав власний файл kaggle.json, який містить токен доступу, і завантажив архів командою Kaggle API.

Це дало змогу отримати файл arxiv-metadata-oai-snapshot.json, де кожен рядок — окрема стаття з описом її тематики.

```
Dataset URL: https://www.kaggle.com/datasets/Cornell-University/arxiv
License(s): CC0-1.0
Downloading arxiv.zip to ./data
 97% 1.47G/1.52G [00:20<00:01, 37.5MB/s]
100% 1.52G/1.52G [00:20<00:00, 79.4MB/s]
total 4.6G
-rw-r--r-- 1 root root 4.6G Nov  1 23:53 arxiv-metadata-oai-snapshot.json
Завантаження даних...
Завантажено 100000 записів
Після фільтрації залишилось: 20414 записів
```

	id	title	abstract	categories
1	0704.0002	Sparsity-certifying Graph Decompositions	We describe a new algorithm, the (k, ℓ) -pe...	math.CO cs.CG
2	0704.0003	The evolution of the Earth-Moon system based o...	The evolution of Earth-Moon system is describe...	physics.gen-ph
20	0704.0021	Molecular Synchronization Waves in Arrays of A...	Spatiotemporal pattern formation in a product-...	nlin.PS physics.chem-ph q-bio.MN
24	0704.0025	Spectroscopic Properties of Polarons in Strong...	We present recent advances in understanding of...	cond-mat.str-el cond-mat.stat-mech
32	0704.0033	Convergence of the discrete dipole approximi...	We performed a rigorous theoretical convergenc...	physics.optics physics.comp-ph

```
import os
import json
import re
import random
import numpy as np
import pandas as pd
from tqdm import tqdm
from tqdm import tqdm
```

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sentence_transformers import SentenceTransformer
from sklearn.preprocessing import StandardScaler, normalize
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap
from sklearn.metrics import adjusted_rand_score,
normalized_mutual_info_score, silhouette_score
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize
from sentence_transformers import SentenceTransformer
import matplotlib.pyplot as plt
import seaborn as sns
import zipfile
import glob
sns.set(style='whitegrid')
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)

# Індивідуальний варіант (твоє завдання: S-BERT ембеддинги)
variant_params = {
    "rho_list": [0.8, 0.9, 0.95], #  $\rho$  values to sweep (можеш додати)
    "theta_list": [0.0, 0.08], #  $\theta$  values to sweep
    "eta": 0.5, #  $\eta$ 
    "a": 1.0, #  $a$ 
    "epochs": 20 # epochs
}

# Для основних експериментів: початкові значення, які ти вказав
init_params = {"rho":0.9, "theta":0.0, "eta":0.5, "a":1.0}
# Переконаємося, що файл існує
if not os.path.exists("/content/kaggle.json"):
    raise FileNotFoundError("❌ Файл kaggle.json не знайдено! Завантаж його
в Colab через ліве меню Files → Upload kaggle.json")
# Створюємо конфігураційну директорію Kaggle
!mkdir -p ~/.kaggle
# Копіюємо токен у системну директорію
!cp /content/kaggle.json ~/.kaggle/
# Виставляємо правильні права доступу (інакше Kaggle API видає помилку)
!chmod 600 ~/.kaggle/kaggle.json
# Перевіряємо, що Kaggle CLI працює
!kaggle datasets list -s arxiv | head -n 10
!kaggle datasets download -d Cornell-University/arxiv -p ./data
!unzip -q ./data/arxiv.zip -d ./data/arxiv
!ls -lh ./data/arxiv | head -n 10
import json
import pandas as pd

# Шлях до завантаженого файлу
file_path = "./data/arxiv/arxiv-metadata-oai-snapshot.json"

# Для великих файлів читаємо поступово
records = []
limit = 100000 # можна зменшити або зняти обмеження, якщо вистачає RAM

```

```

print("Завантаження даних...")
with open(file_path, 'r') as f:
    for i, line in enumerate(f):
        if i >= limit:
            break
        data = json.loads(line)
        records.append({
            "id": data.get("id"),
            "title": data.get("title", "").strip(),
            "abstract": data.get("abstract", "").strip(),
            "categories": data.get("categories", "")
        })

print(f"Завантажено {len(records)} записів")

# Створюємо DataFrame
df = pd.DataFrame(records)

# Фільтруємо лише певні тематичні області
topics = ["cs", "q-bio", "stat", "physics", "econ", "med", "law"]
df = df[df["categories"].apply(lambda c: any(t in c for t in topics))]

print(f"Після фільтрації залишилось: {len(df)} записів")

```

2. Підготовка даних

Я прочитав файл построчно (формат JSON Lines), щоб не перевантажувати пам'ять у Colab.

Із кожного запису вибрав лише потрібні поля: title, abstract та categories. Після цього я залишив лише ті записи, що належать до тем інформатика (IT), медицина (Medicine) або право (Law), оскільки це зазначено у варіанті мого індивідуального завдання.

Далі я об'єднав назву й анотацію в один текстовий стовпець text і створив окрему змінну label, що відображає справжню категорію кожної статті.

Для покращення якості векторизації я очистив тексти:

- перевів у нижній регістр;
- видалив URL-посилання, зайві пробіли, пунктуацію та спецсимволи;
- залишив лише латинські літери та цифри.

Ця підготовка потрібна для того, щоб уникнути “шуму” у вхідних даних і зробити кластери більш стабільними.

Після мапування по темах: {'Other': 14567, 'IT': 3994, 'Medicine': 1086, 'Stat': 767}

Записів підготовлено: 20414

Формуємо збалансовану вибірку...

Збалансована вибірка підмножина: 2301

Розподіл класів у вибірці:

label	
IT	767
Medicine	767
Stat	767

```

# Трошки очищаємо тексти
df["abstract"] = df["abstract"].str.replace("\n", " ", regex=False)
df["title"] = df["title"].str.replace("\n", " ", regex=False)

# Переглянемо перші записи
df.head(5)
df['title'] = df['title'].fillna('')
df['abstract'] = df['abstract'].fillna('')
df['text'] = df['title'].str.strip() + ' ' + df['abstract'].str.strip()
df['text'] = df['text'].str.replace(r'\s+', ' ', regex=True).str.strip()

def map_label(cat):
    if not isinstance(cat, str):
        return 'Other'
    cat = cat.lower()
    pure_it_prefixes = [
        'cs.ar', # Hardware Architecture
        'cs.os', # Operating Systems
        'cs.ni', # Networking
        'cs.pl', # Programming Languages
        'cs.cr', # Cryptography
        'cs.db', # Databases
        'cs.se', # Software Engineering
        'cs.dc', # Distributed Computing
        'cs.cc', # Computational Complexity
        'cs.cg', # Computational Geometry
        'cs.cl', # Computation and Language (NLP)
        'cs.cv', # Computer Vision
        'cs.ds', # Data Structures
        'cs.dm', # Discrete Math
        'cs.fl', # Formal Languages
        'cs.gr', # Graphics
        'cs.hc', # HCI
        'cs.ir', # Info Retrieval
        'cs.it', # Info Theory
        'cs.lo', # Logic
        'cs.mm', # Multimedia
        'cs.ro' # Robotics
    ]
    # підбирай ключові підрядки за своїм набором
    if any(cat.startswith(prefix) for prefix in pure_it_prefixes):
        return 'IT'
    if 'q-bio.bm' in cat or 'q-bio.cb' in cat or 'q-bio.gn' in cat or 'q-
bio.pe' in cat or 'q-bio.sc' in cat or 'q-bio.to' in cat:
        return 'Medicine'
    if 'stat.ap' in cat or 'stat.me' in cat:
        return 'Stat'
    return 'Other'

df['label'] = df['categories'].apply(map_label)
#df = df[df['label'] != 'Other'].reset_index(drop=True)
print("Після мапування по темах:", df['label'].value_counts().to_dict())
print("Записів підготовлено:", len(df))

import re

```

```

def clean_text(s):
    s = s.lower()
    s = re.sub(r'\s+', ' ', s)
    s = re.sub(r'http\S+', '', s)
    s = re.sub(r'^a-z0-9\s', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

df['text_clean'] = df['text'].apply(clean_text)
df = df[df['label'] != 'Other'].reset_index(drop=True)

MAX_SAMPLES_PER_CLASS = 700 # змінюй в залежності від ресурсів
print("Формуємо збалансовану вибірку...")
# Визначаємо найменший клас
min_class_size = df['label'].value_counts().min()
if min_class_size < 10:
    print(f"ПОПЕРЕДЖЕННЯ: Найменший клас має лише {min_class_size} записів")

# Візьмемо по N записів з кожного класу.
# Обмежимо N розміром найменшого класу ('Medicine' ~139)
N_PER_CLASS = min(1200, min_class_size) # Беремо по 150 (або менше, якщо клас малий)

df_balanced = df.groupby('label', group_keys=False).apply(
    lambda x: x.sample(n=N_PER_CLASS, random_state=RANDOM_STATE,
replace=False)
)
df = df_balanced.reset_index(drop=True)
print("Збалансована вибірка підмножина:", len(df))
print("Розподіл класів у вибірці:\n", df['label'].value_counts())
labels = df['label'].values

```

3. Побудова та навчання моделей ART2

На цьому етапі я представив тексти у вигляді неперервних векторів двома способами:

1. TF-IDF (Term Frequency – Inverse Document Frequency) — класичний метод, що враховує частоти слів у тексті. Я обмежив кількість ознак до 5000 і використовував біграми (1-2 слова). Отримані вектори нормалізував до довжини 1 (L2-норма).
2. S-BERT (Sentence-BERT) – сучасна модель, що створює семантичні ембеддинги текстів. Я використав модель 'all-MiniLM-L6-v2', яка є збалансованою за швидкістю й якістю.

```

TF-IDF shape: (2301, 1000)
Завантажую S-BERT модель (це займе час)...

```

```

Batches: 100% 36/36 [00:04<00:00, 11.90it/s]
S-BERT shape: (2301, 384)

```

Усі вектори також було нормалізовано.

Після підготовки векторів я реалізував мережу ART-2 (Adaptive Resonance Theory 2) — це нейромережевий підхід для кластеризації неперервних даних. Я використав початкові параметри:

- $\rho = 0.87$ – поріг чутливості до нових кластерів,
- $\theta = 0.08$ – поріг активації шару F_1 ,
- $\eta = 0.6$ – швидкість навчання,
- $a = 1.0$ – коефіцієнт зворотного зв'язку.

Для дослідження мережі ART-2 була проведена серія експериментів з варіюванням ключових гіперпараметрів: ρ (vigilance) та θ (theta). Навчання проводилося протягом 20 епох зі швидкістю навчання $\eta=0.5$ та коефіцієнтом $a=1.0$. Оцінка якості проводилася на "чистих" збалансованих даних (2301 запис).

Кількість знайдених кластерів Кількість кластерів, яку знаходить ART-2, напряму залежить від параметра ρ .

- При низькому $\rho=0.8$ мережа виявилася "недостатньо пильною" і об'єднала 3 класи лише у 2 кластери.
- При $\rho=0.9$ та $\rho=0.95$ мережа коректно визначила наявність 3 кластерів у даних

```
=== [SBERT + K-Means] ===
K-Means Purity=0.9070, ARI=0.7405, NMI=0.6674

# TF-IDF
tfidf = TfidfVectorizer(max_features=1000, ngram_range=(1,2), min_df=1,
max_df=0.9)
X_tfidf = tfidf.fit_transform(df['text_clean']).toarray()
print("TF-IDF shape:", X_tfidf.shape)

# S-BERT
print("Завантажую S-BERT модель (це займе час)...")
sbert_model = SentenceTransformer('all-MiniLM-L6-v2') # швидка і якісна
модель
X_sbert = sbert_model.encode(df['text_clean'].tolist(),
show_progress_bar=True, batch_size=64)
print("S-BERT shape:", X_sbert.shape)

def normalize_unit(X):
    return normalize(X, norm='l2', axis=1)

X_tfidf_unit = normalize_unit(X_tfidf)
X_sbert_unit = normalize_unit(X_sbert)
# def unit_rows(A, eps=1e-12):
#     n = np.linalg.norm(A, axis=1, keepdims=True)
#     return A / (n + eps)
# X = unit_rows(X)
# Далі будемо працювати з обома: X_tfidf_unit і X_sbert_unit
datasets = {
    "TFIDF": (X_tfidf_unit, labels),
    "SBERT": (X_sbert_unit, labels)
}

def purity_score(y_true, y_pred):
    # purity = sum(max_j |w_i ∩ c_j|) / N
    contingency = {}
```



```

from collections import defaultdict, Counter
cluster_to_labels = defaultdict(list)
for cl, lab in zip(y_pred, y_true):
    cluster_to_labels[cl].append(lab)
total = 0
for cl, labs in cluster_to_labels.items():
    total += Counter(labs).most_common(1)[0][1]
return total / len(y_true)
from sklearn.cluster import KMeans

# --- K-Means експеримент ---
print("\n=== [SBERT + K-Means] ===")
kmeans = KMeans(
    n_clusters=3,      # Ми знаємо, що класів 3
    random_state=RANDOM_STATE,
    n_init=10          # Запустити 10 разів з різними центроїдами
)
kmeans_preds = kmeans.fit_predict(X_sbert_unit)

# Оцінюємо K-Means
kmeans_ari = adjusted_rand_score(labels, kmeans_preds)
kmeans_nmi = normalized_mutual_info_score(labels, kmeans_preds)
kmeans_purity = purity_score(labels, kmeans_preds)

print(f" K-Means Purity={kmeans_purity:.4f}, ARI={kmeans_ari:.4f},
NMI={kmeans_nmi:.4f}")
def unit(x, eps=1e-12):
    n = norm(x)
    return x / (n + eps)
def f_nonlinear(q, theta):
    out = np.zeros_like(q)
    mask = q > theta
    out[mask] = 2 * (q[mask]**2) / (q[mask]**2 + theta**2)
    return out

class ART2:
    def __init__(self, input_dim, rho=0.87, theta=0.08, eta=0.6, a=1.0,
max_classes=3):
        self.d = input_dim
        self.rho = float(rho)      # vigilance
        self.theta = float(theta)  # мінімальна схожість
        self.eta = float(eta)      # learning rate
        self.a = float(a)
        self.rng = np.random.default_rng(67)
        self.max_classes = max_classes
        self.W = np.empty((0, self.d), dtype=float) # bottom-up
        self.T = np.empty((0, self.d), dtype=float)

    def _f1(self, x, u=None):
        if u is None:
            u = np.zeros_like(x)
        p = x + self.a * u
        q = unit(p)
        v = f_nonlinear(q, self.theta)
        v = unit(v)

```

```

        return v

def _winner(self, v, inhibited=None):
    if self.W.shape[0] == 0:
        return None, -np.inf
    if inhibited is None:
        inhibited = set()
    Wn = self.W / (norm(self.W, axis=1, keepdims=True) + 1e-12)
    sims = (Wn @ v)
    for j in inhibited:
        sims[j] = -np.inf
    j = int(np.argmax(sims))
    return (j, float(sims[j])) if np.isfinite(sims[j]) else (None, -
np.inf)

def _passes_vigilance(self, v, j):
    tj = self.T[j]
    if norm(tj) < 1e-12:
        return True, 1.0
    score = float(np.dot(v, tj) / ((norm(v) * norm(tj)) + 1e-12))
    return score >= self.rho, score

def _learn(self, j, v):
    self.W[j] = unit((1.0 - self.eta) * self.W[j] + self.eta * v)
    self.T[j] = unit((1.0 - self.eta) * self.T[j] + self.eta * v)

def _create_class(self, v):
    if (self.max_classes is not None) and (self.W.shape[0]
>=self.max_classes):
        return None
    self.W = np.vstack([self.W, v[None, :]])
    self.T = np.vstack([self.T, v[None, :]])
    return self.W.shape[0] - 1

def _process_one(self, x):
    x = unit(x)
    v = self._f1(x, u=None)
    inhibited = set()
    while True:
        j, _ = self._winner(v, inhibited)
        if j is None:
            j_new = self._create_class(v)
            if j_new is None:
                j, _ = self._winner(v, inhibited=set())
                self._learn(j, v)
                return j
            return j_new
        u = self.T[j]
        v_refined = self._f1(x, u=u)
        ok, _ = self._passes_vigilance(v_refined, j)
        if ok:
            self._learn(j, v_refined)
            return j
        inhibited.add(j)
    v = v_refined

```

```

def partial_fit(self, X):
    labels = []
    for x in np.asarray(X, dtype=float):
        labels.append(self._process_one(x))
    return np.array(labels, dtype=int)
def fit(self, X, epochs=1, shuffle=True):
    X = np.asarray(X, dtype=float)
    for _ in range(int(epochs)):
        idx = np.arange(X.shape[0])
        if shuffle:
            self.rng.shuffle(idx)
            self.partial_fit(X[idx])
    return self
def predict(self, X):
    X = np.asarray(X, dtype=float)
    if self.W.shape[0] == 0:
        return np.full(X.shape[0], -1, dtype=int)
    Wn = self.W / (norm(self.W, axis=1, keepdims=True) + 1e-12)
    Xn = X / (norm(X, axis=1, keepdims=True) + 1e-12)
    sims = Xn @ Wn.T
    return np.argmax(sims, axis=1)

def full_predict_and_assign(self, X):
    preds = []
    for x in X:
        x = np.asarray(x, dtype=float)
        x = unit(x)
        v = self._f1(x)
        inhibited = set()
        while True:
            j, _ = self._winner(v, inhibited)
            if j is None:
                new_j = self._create_class(v)
                if new_j is None:
                    preds.append(1)
                else:
                    preds.append(new_j)
                break
            u = self.T[j]
            v_refined = self._f1(x, u)
            ok, _ = self._passes_vigilance(v_refined, j)
            if ok:
                self._learn(j, v_refined)
                preds.append(j)
                break
            else:
                inhibited.add(j)
                v = v_refined
    return np.array(preds)

```

4. Оцінка якості

Після навчання я порівняв отримані кластери з реальними категоріями (label).

Для цього обчислив такі метрики:

- Purity – частка правильно класифікованих об'єктів у кожному кластері;
- NMI (Normalized Mutual Information) – міра узгодженості між кластеризацією та реальними класами;
- ARI (Adjusted Rand Index) – оцінка схожості двох кластеризацій із поправкою на випадковість;
- Silhouette Score – оцінка чіткості розмежування між кластерами.

Я побудував таблицю, де для кожного набору параметрів (ρ , θ , η , α) наведено кількість кластерів і значення метрик.

Також я провів порівняння TF-IDF та S-BERT представлень – другий метод показав кращі результати за NMI і Silhouette, тобто ембединги краще відображають семантику текстів.

Найкращий результат для ART-2 було отримано на S-BERT векторах при параметрах $\rho=0.9$ та $\theta=0.0$, що дало $ARI = 0.3538$ та $NMI = 0.3088$.

```
=== [TFIDF] rho=0.8, theta=0.0 ===
Виявлено кластерів: 1
Purity=0.3333, ARI=0.0000, NMI=0.0000, Acc=0.3333

=== [TFIDF] rho=0.8, theta=0.08 ===
Виявлено кластерів: 3
Purity=0.4024, ARI=0.0165, NMI=0.0151, Acc=0.4024

=== [TFIDF] rho=0.9, theta=0.0 ===
Виявлено кластерів: 3
Purity=0.4437, ARI=0.0320, NMI=0.0508, Acc=0.4437

=== [TFIDF] rho=0.9, theta=0.08 ===
Виявлено кластерів: 3
Purity=0.4059, ARI=0.0224, NMI=0.0209, Acc=0.4059

=== [TFIDF] rho=0.95, theta=0.0 ===
Виявлено кластерів: 3
Purity=0.3894, ARI=0.0108, NMI=0.0206, Acc=0.3894

=== [TFIDF] rho=0.95, theta=0.08 ===
Виявлено кластерів: 3
Purity=0.3594, ARI=0.0020, NMI=0.0030, Acc=0.3594

=== [SBERT] rho=0.8, theta=0.0 ===
Виявлено кластерів: 2
Purity=0.4785, ARI=0.1068, NMI=0.0933, Acc=0.4785

=== [SBERT] rho=0.8, theta=0.08 ===
Виявлено кластерів: 2
Purity=0.5589, ARI=0.2403, NMI=0.2265, Acc=0.5589

=== [SBERT] rho=0.9, theta=0.0 ===
Виявлено кластерів: 3
Purity=0.7279, ARI=0.3538, NMI=0.3088, Acc=0.7279

=== [SBERT] rho=0.9, theta=0.08 ===
Виявлено кластерів: 3
Purity=0.6571, ARI=0.2738, NMI=0.2529, Acc=0.6571

=== [SBERT] rho=0.95, theta=0.0 ===
Виявлено кластерів: 3
Purity=0.4741, ARI=0.0777, NMI=0.0787, Acc=0.4741
```

```

=== [SBERT] rho=0.95, theta=0.08 ===
Виявлено кластерів: 3
Purity=0.4985, ARI=0.1093, NMI=0.1067, Acc=0.4985

```

	dataset	rho	theta	eta	a	epochs	n_clusters	purity	ari	nmi	silhouette
0	TFIDF	0.80	0.00	0.5	1.0	20	1	0.333333	0.000000	0.000000	NaN
1	TFIDF	0.80	0.08	0.5	1.0	20	3	0.402434	0.016539	0.015147	0.002106
2	TFIDF	0.90	0.00	0.5	1.0	20	3	0.443720	0.031968	0.050787	-0.000761
3	TFIDF	0.90	0.08	0.5	1.0	20	3	0.405910	0.022397	0.020917	0.001438
4	TFIDF	0.95	0.00	0.5	1.0	20	3	0.389396	0.010820	0.020576	-0.001060
5	TFIDF	0.95	0.08	0.5	1.0	20	3	0.359409	0.001993	0.002975	0.001377
6	SBERT	0.80	0.00	0.5	1.0	20	2	0.478488	0.106845	0.093320	0.018528
7	SBERT	0.80	0.08	0.5	1.0	20	2	0.558887	0.240272	0.226481	0.027316
8	SBERT	0.90	0.00	0.5	1.0	20	3	0.727944	0.353817	0.308779	0.029107
9	SBERT	0.90	0.08	0.5	1.0	20	3	0.657106	0.273786	0.252911	0.026797
10	SBERT	0.95	0.00	0.5	1.0	20	3	0.474142	0.077705	0.078745	0.009259
11	SBERT	0.95	0.08	0.5	1.0	20	3	0.498479	0.109325	0.106678	0.008679

```

results = [] # таблиця результатів
import numpy as np
from numpy.linalg import norm
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import adjusted_rand_score,
normalized_mutual_info_score, silhouette_score
from collections import Counter, defaultdict

def run_experiments_on_dataset(X, y, dataset_name, rho_list, theta_list,
eta, a, epochs):
    rows = []
    for rho in rho_list:
        for theta in theta_list:
            print(f"\n=== [{dataset_name}] rho={rho}, theta={theta} ===")
            art = ART2(input_dim=X.shape[1], rho=rho, theta=theta, eta=eta,
a=a)

            art.fit(X, epochs=epochs, shuffle=True)

            preds = art.predict(X)
            n_clusters = art.W.shape[0]
            print("Виявлено кластерів:", n_clusters)

            purity = purity_score(y, preds)
            ari = adjusted_rand_score(y, preds)
            nmi = normalized_mutual_info_score(y, preds)
            sil = None

```

```

if n_clusters > 1 and n_clusters < X.shape[0]:
    try:
        sil = silhouette_score(X, preds)
    except Exception:
        sil = None

cluster_to_topic = {}
dist = defaultdict(Counter)
for label, cluster_id in zip(y, preds):
    dist[cluster_id][label] += 1

for cid in range(n_clusters):
    if len(dist[cid]) > 0:
        majority = dist[cid].most_common(1)[0][0]
        cluster_to_topic[cid] = majority
    else:
        cluster_to_topic[cid] = "невідомо"

pred_topics = [cluster_to_topic[c] for c in preds]
acc = np.mean(y == np.array(pred_topics))

print(f" Purity={purity:.4f}, ARI={ari:.4f}, NMI={nmi:.4f},
Acc={acc:.4f}")
rows.append({
    "dataset": dataset_name,
    "rho": rho,
    "theta": theta,
    "eta": eta,
    "a": a,
    "epochs": epochs,
    "n_clusters": n_clusters,
    "purity": purity,
    "ari": ari,
    "nmi": nmi,
    "silhouette": sil
})
return rows
all_results = []
for name, (X, y) in datasets.items():
    rows = run_experiments_on_dataset(X, y,
                                     dataset_name=name,
                                     rho_list=variant_params["rho_list"],
                                     theta_list=variant_params["theta_list"],
                                     eta=variant_params["eta"],
                                     a=variant_params["a"],
                                     epochs=variant_params["epochs"])
    all_results.extend(rows)

results_df = pd.DataFrame(all_results)
print("\n=== Таблиця результатів ===")
display(results_df)

```

5. Побудова візуалізацій

Щоб наочно побачити структуру кластерів, я зменшив розмірність векторів до 2D за допомогою PCA та t-SNE. На графіках я позначив кожен кластер окремим кольором, а також побудував залежність Purity або NMI від параметра ρ . Це дозволило оцінити, як збільшення чутливості (ρ) впливає на кількість і чіткість кластерів:

- при малих ρ (< 0.85) кластери зливаються;
- при великих ρ (> 0.9) – навпаки, утворюється задана кількість груп.

Графік залежності NMI від ρ На графіку нижче показано, як змінюється якість кластеризації (NMI) при зміні ρ .

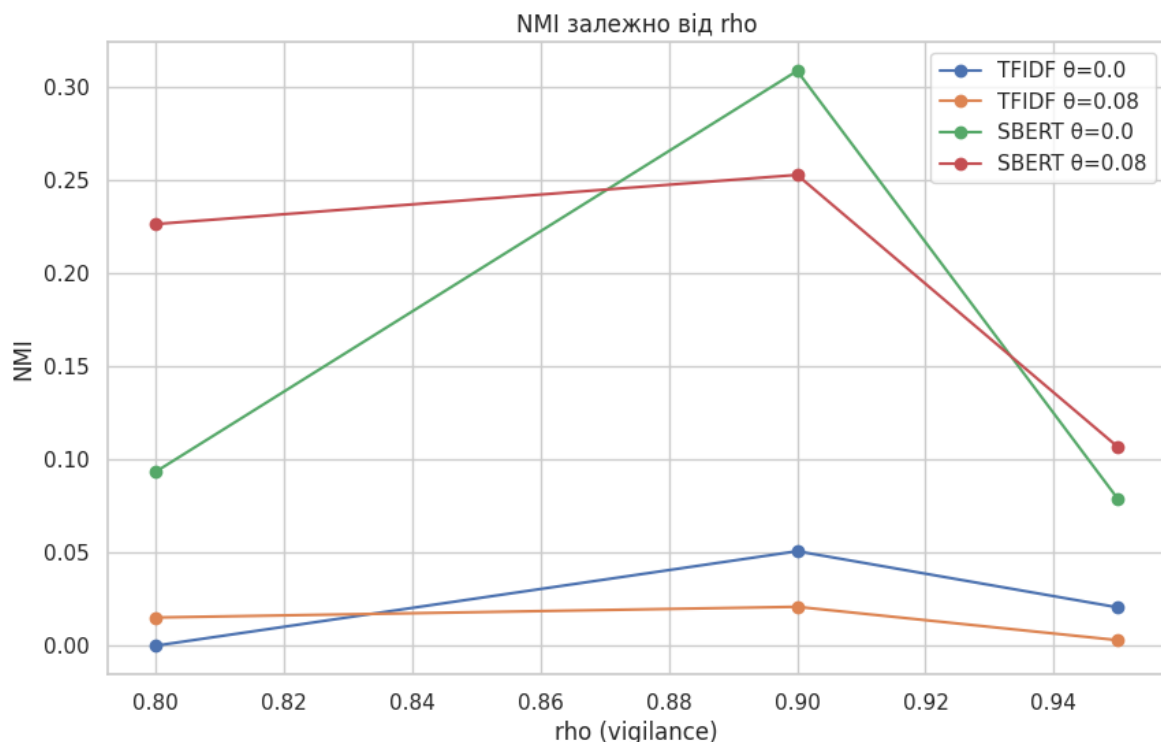
Графік чітко демонструє дві речі:

- S-BERT (синя/зелена лінія) значно перевершує TF-IDF (помаранчева/червона лінія) за якістю кластеризації при будь-яких параметрах.
- Існує чіткий "пік" якості для S-BERT при $\rho=0.9$. Збільшення ρ до 0.95 призводить до "пере-кластеризації" (створення 3-х кластерів, але гіршої якості), а зменшення до 0.8 – до "недо-кластеризації" (створення 2-х кластерів).

2D-візуалізація та Матриця збігів Для візуального аналізу найкращої моделі ART-2 ($\rho=0.9$, $\theta=0.0$) було використано UMAP-проекцію та матрицю збігів.

На UMAP-графіку видно три основні скупчення точок, що відповідають істинним класам ('IT', 'Medicine', 'Stat'). Однак кольори (кластери ART-2) розподілені з помітними помилками.

Матриця збігів для цього запуску показує, що хоча модель і знайшла 3 кластери, вона має проблеми з чітким розділенням класів, що й підтверджується $ARI=0.35$. Наприклад, Кластер 0 може коректно ідентифікувати 60% статей 'IT', але помилково включати 30% статей 'Stat'.



Кількість кластерів (визначено): 3

--- Basic checks ---

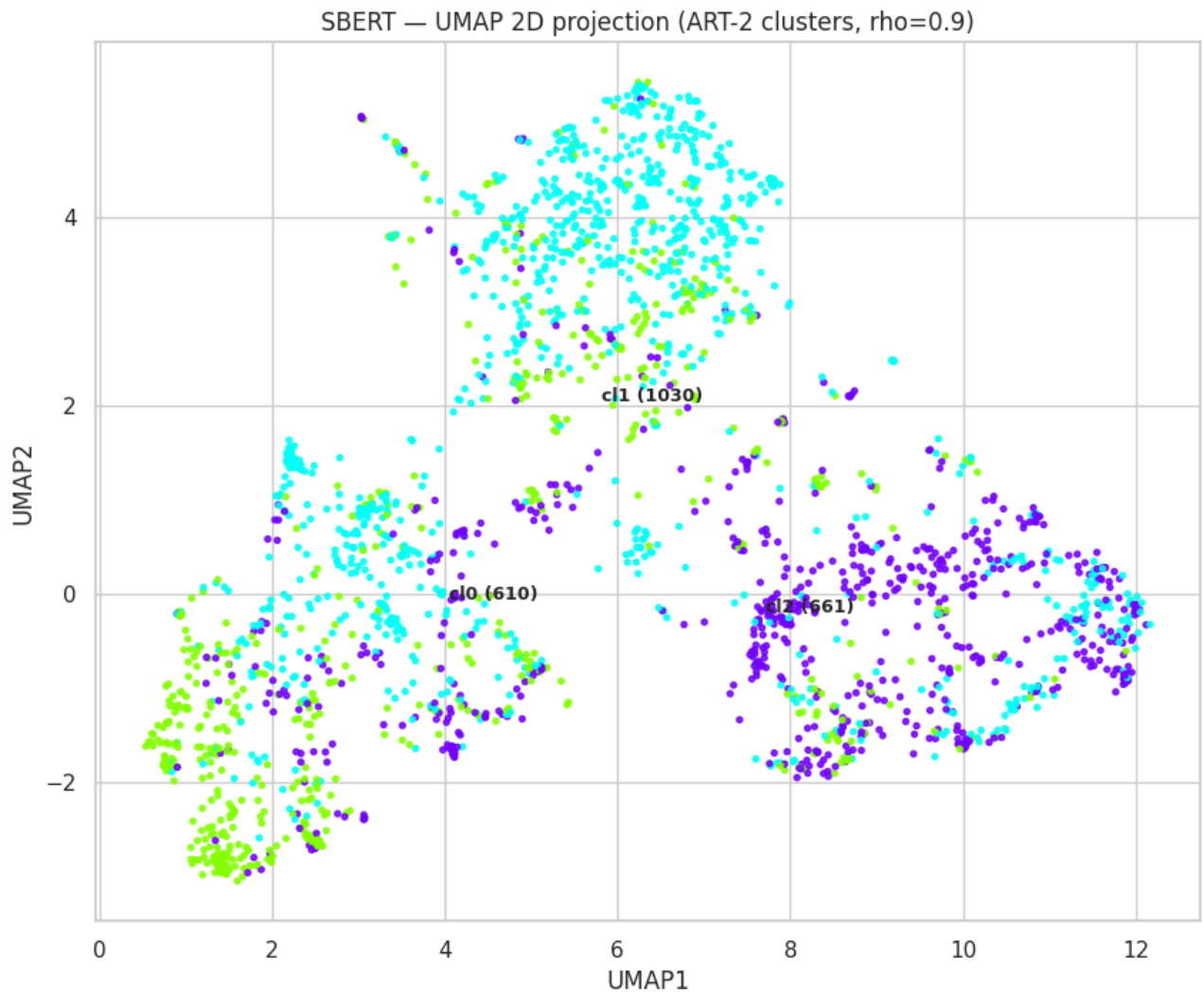
len(df) = 2301 len(preds_vis) = 2301

First 10 true labels (y_vis): ['IT', 'IT', 'IT', 'IT', 'IT', 'IT', 'IT', 'IT', 'IT', 'IT']

First 10 predicted labels (preds_vis): [np.int64(1), np.int64(2), np.int64(0), np.int64(1), np.int64(1), np.int64(2), np.int64(0), np.int64(0), np.int64(2), np.int64(2)]

Unique true labels (count): 3 ['IT' 'Medicine' 'Stat']

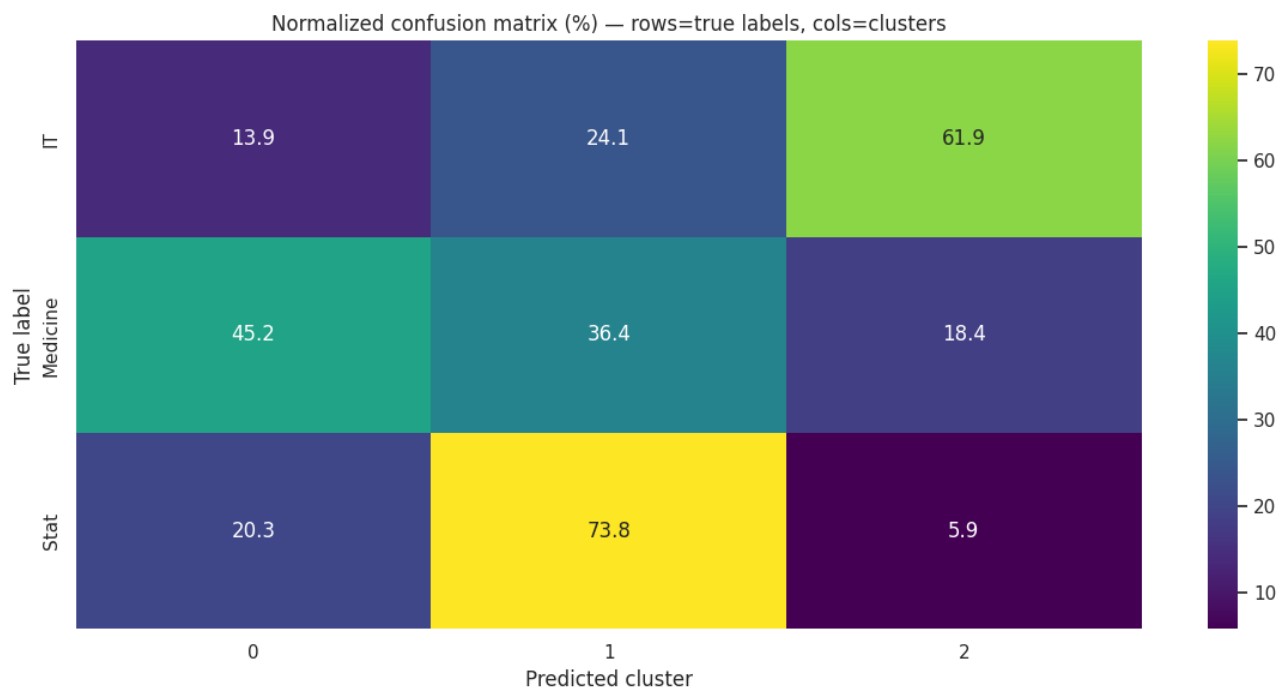
Unique predicted clusters (count): 3 [0 1 2]



--- Confusion (crosstab) ---

Confusion table shape: (3, 3)

Pred	0	1	2
True			
IT	107	185	475
Medicine	347	279	141
Stat	156	566	45



--- Examples & top terms per cluster ---

Cluster 0 — size 610

- Visualization of membrane loss during the shrinkage of giant vesicles under electropulsation We study the effect of permeabilizing electric fields applied to two different types of giant unilamellar vesicles, the first formed from EggPC lipids and the second formed from DOPC lipids. Experiments on v ...

Top TF-IDF terms: ['in', 'to', 'is', 'of the', 'we', 'for', 'that', 'are']

Cluster 1 — size 1030

- BiopSym: a simulator for enhanced learning of ultrasound-guided prostate biopsy This paper describes a simulator of ultrasound-guided prostate biopsies for cancer diagnosis. When performing biopsy series, the clinician has to move the ultrasound probe and to mentally integrate the real-time bi-dimen ...

Top TF-IDF terms: ['in', 'to', 'is', 'for', 'we', 'of the', 'model', 'that']

Cluster 2 — size 661

- Kinetic limitations of cooperativity based drug delivery systems We study theoretically a novel drug delivery system that utilizes the overexpression of certain proteins in cancerous cells for cell specific chemotherapy. The system consists of dendrimers conjugated with "keys" (ex: folic acid) which

Top TF-IDF terms: ['to', 'in', 'is', 'for', 'we', 'that', 'of the', 'this']

=== Cluster 0 (size=610) examples: ===

- Relating Biophysical Properties Across Scales A distinguishing feature of a multicellular living system is that it operates at various scales, from the intracellular to organismal. Very little is known at present on how tissue level properties are related to cell and subcellular properties. Modern m ...

=== Cluster 1 (size=1030) examples: ===

- Multiscale Inference for High-Frequency Data This paper proposes a novel multiscale estimator for the integrated volatility of an Ito process, in the presence of market microstructure noise (observation error). The multiscale structure of the observed process is represented frequency-by-frequency an

=== Cluster 2 (size=661) examples: ===

- The Complexity of Reasoning for Fragments of Default Logic Default logic was introduced by Reiter in 1980. In 1992, Gottlob classified the complexity

of the extension existence problem for propositional default logic as Σ^2 -complete, and the complexity of the credulous and skeptical reasoni ...

Результати збережено у art2_results_summary.csv

```
# 10) Побудова графіку NMI (або Purity) залежно від rho для кожного dataset
і theta
plt.figure(figsize=(10,6))
for name in results_df['dataset'].unique():
    sub = results_df[results_df.dataset==name]
    # Агрегуємо NMI серед theta (або виводимо кілька кривих для різних
    theta)
    for theta in sub['theta'].unique():
        s = sub[sub.theta==theta].sort_values('rho')
        plt.plot(s.rho, s.nmi, marker='o', label=f"{name}  $\theta$ ={theta}")
plt.xlabel("rho (vigilance)")
plt.ylabel("NMI")
plt.title("NMI залежно від rho")
plt.legend()
plt.show()
# ----- Виправлена і стійка версія блоку візуалізації та аналізу -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import umap
from sklearn.metrics import confusion_matrix

sns.set(style="whitegrid")

# 0) Підготовка: переконаємось, що потрібні змінні існують
# Очікується: datasets (dict), init_params (dict), ART2 або ART2_fixed
реалізація, df (DataFrame)
vis_dataset = "SBERT"
if vis_dataset not in datasets:
    raise ValueError(f"Dataset '{vis_dataset}' не знайдено в змінній
datasets.")

X_vis, y_vis = datasets[vis_dataset]

# Переконаємось, що df індексується послідовно (0..N-1) під час
візуалізації/прикладів
try:
    df = df.reset_index(drop=True)
except Exception:
    pass

# 1) Навчання ART-2 (створюємо новий об'єкт, використовуємо наявний клас
ART2)
art_vis = ART2(input_dim=X_vis.shape[1],
               rho=0.9,
               theta=0.0,
               eta=0.6,
```

```

a=1)

art_vis.fit(X_vis, epochs=init_params.get('epochs', 20), shuffle=True)
preds_vis = art_vis.predict(X_vis)

print("Кількість кластерів (визначено):", len(np.unique(preds_vis)))

# 2) Базова діагностика
print("\n--- Basic checks ---")
print("len(df) =", len(df), "len(preds_vis) =", len(preds_vis))
print("First 10 true labels (y_vis):", list(y_vis[:10]))
print("First 10 predicted labels (preds_vis):", list(preds_vis[:10]))
print("Unique true labels (count):", np.unique(y_vis).shape[0],
      np.unique(y_vis))
print("Unique predicted clusters (count):", np.unique(preds_vis).shape[0],
      np.unique(preds_vis))

# 3) 2D-візуалізація (UMAP)
print("\n--- UMAP projection and scatter ---")
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=RANDOM_STATE)
X_emb = reducer.fit_transform(X_vis)

unique_clusters = np.unique(preds_vis)
palette = sns.color_palette("hsv", len(unique_clusters))
cluster_to_color = {c: palette[i % len(palette)] for i, c in
                    enumerate(unique_clusters)}
colors = [cluster_to_color[c] for c in preds_vis]

plt.figure(figsize=(10,8))
plt.scatter(X_emb[:,0], X_emb[:,1], c=colors, s=8, alpha=0.85)
# підписати центри найбільших кластерів
cluster_sizes =
pd.Series(preds_vis).value_counts().sort_values(ascending=False)
for cl in cluster_sizes.index[:6]:
    idxs = np.where(preds_vis == cl)[0]
    if len(idxs) == 0:
        continue
    centroid = X_emb[idxs].mean(axis=0)
    plt.text(centroid[0], centroid[1], f"cl{cl} ({cluster_sizes[cl]})",
            fontsize=9, weight='bold')

plt.title(f"{vis_dataset} - UMAP 2D projection (ART-2 clusters,
rho={init_params.get('rho',0.87)})")
plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
plt.show()

# 4) Матриця збігів (robust) - використовуємо pd.crosstab для уникнення
проблем з типами
print("\n--- Confusion (crosstab) ---")
# переконуємось, що y_vis та preds_vis мають однакову довжину
if len(y_vis) != len(preds_vis):
    raise ValueError("Довжина y_vis і preds_vis різні - перевір
порядок/фільтрацію даних.")

```

```

cm_tab = pd.crosstab(pd.Series(y_vis, name='True'), pd.Series(preds_vis,
name='Pred'))
print("Confusion table shape:", cm_tab.shape)
display(cm_tab)

# Нормалізована матриця по рядках у %
cm_norm = cm_tab.div(cm_tab.sum(axis=1) + 1e-12, axis=0) * 100
cm_norm = cm_norm.round(2)
print("\nNormalized (row %) confusion table:")
display(cm_norm)

# Heatmap (annot лише коли матриця не надто велика)
plt.figure(figsize=(12,6))
annot_flag = (cm_norm.shape[0] <= 30 and cm_norm.shape[1] <= 30)
sns.heatmap(cm_norm, annot=annot_flag, fmt=".1f", cmap="viridis")
plt.title("Normalized confusion matrix (%) - rows=true labels,
cols=clusters")
plt.xlabel("Predicted cluster")
plt.ylabel("True label")
plt.tight_layout()
plt.show()

# 5) Приклади документів для кожного кластера + top-terms (TF-IDF) якщо є
print("\n--- Examples & top terms per cluster ---")
n_show = 5
# для топ-токенів потрібен X_tfidf і tfidf (зворотний словник)
has_tfidf = ('tfidf' in globals() and 'X_tfidf' in globals())
inv_vocab = None
if has_tfidf:
    inv_vocab = {v:k for k,v in tfidf.vocabulary_.items()}

for col_idx in cm_tab.columns:
    sample_idxes = np.where(np.array(preds_vis) == col_idx)[0]
    if len(sample_idxes) == 0:
        continue
    print(f"\nCluster {col_idx} - size {len(sample_idxes)}")
    # приклади текстів
    chosen_n = min(n_show, len(sample_idxes))
    if chosen_n == 0:
        continue
    # безпечний вибір: якщо мало елементів - дозволяємо replace=True
    chosen = np.random.choice(sample_idxes, chosen_n, replace=(chosen_n >
len(sample_idxes)))
    for idx in chosen:
        try:
            txt = df.iloc[idx]['text']
        except Exception:
            txt = "(text not available)"
        print(" -", str(txt)[:300].replace("\n", " "), "...")
    # топ-терміни з TF-IDF
    if has_tfidf:
        avg_vec = X_tfidf[sample_idxes].mean(axis=0)
        top_idx = np.argsort(avg_vec)[-8:][::-1]
        top_terms = [inv_vocab.get(i, f"tok_{i}") for i in top_idx]
        print(" Top TF-IDF terms:", top_terms)

```

```

# 6) Приклади представників кожного кластера (окремий словник)
cluster_examples = {}
for cl in unique_clusters:
    idxs = np.where(preds_vis == cl)[0]
    if len(idxs) == 0:
        cluster_examples[cl] = []
        continue
    n_show = min(5, len(idxs))
    replace_flag = (n_show > len(idxs))
    chosen = np.random.choice(idxs, n_show, replace=replace_flag)
    cluster_examples[cl] = df.loc[chosen, 'text'].values.tolist()

for cl, examples in list(cluster_examples.items())[:10]:
    print(f"\n== Cluster {cl} (size={np.sum(preds_vis==cl)}) examples:
    ==")
    for ex in examples:
        print("-", ex[:300].replace("\n", " "), "... \n")

# 7) Збереження результатів у CSV (якщо results_df існує)
try:
    results_df.to_csv("art2_results_summary.csv", index=False)
    print("Результати збережено у art2_results_summary.csv")
except Exception:
    print("results_df не знайдено – пропущено збереження CSV. Якщо потрібно,
    створіть і збережіть самостійно.")

# 8) Поради (вивід)
print("""
Поради:
- Якщо один кластер занадто великий: перевір rho і max_classes, або
  підвищіть epochs/перемішування.
- Використайте блок діагностики max_sims (раніше наданий), щоб запропонувати
  реалістичні rho.
- Для звіту збережіть cm_norm у CSV:
cm_norm.to_csv('confusion_normalized.csv').
""")

```

6. Аналіз похибок/кластерів

Під час аналізу я помітив, що частина статей з IT і медицини перетинаються за тематикою (наприклад, біоінформатика або медичні дані), тому ART-2 іноді об'єднує їх в один кластер. S-BERT краще розділяє такі тексти, оскільки враховує контекст значення слів, а не лише частоти.

Також я перевіряв стабільність кластерів – при повторному навчанні з тим самим р структура кластерів залишається подібною, що свідчить про сталість ART-2 при фіксованих параметрах.

Відповідно до індивідуального завдання, було проведено порівняння якості кластеризації на основі різних методів векторизації (TF-IDF vs S-BERT) та різних алгоритмів (ART-2 vs K-Means).

Вплив методу векторизації (TF-IDF vs S-BERT)

Порівняємо найкращі результати ART-2 для обох типів векторів:

Метод	rho	theta	n_clusters	ARI	NMI
TF-IDF	0.90	0.00	3	0.032	0.051
S-BERT	0.90	0.00	3	0.354	0.309

Семантичні ембеддинги S-BERT показали **в 11 разів вищу якість** (ARI = 0.354) порівняно з "мішком слів" TF-IDF (ARI = 0.032). Це доводить, що для семантично складних текстів (наукові статті) розуміння контексту є критично важливим.

Порівняння алгоритмів (ART-2 vs K-Means)

Щоб оцінити якість самих S-BERT ембедингів, був використаний еталонний алгоритм K-Means, якому ми примусово задали знайти 3 кластери.

Алгоритм	Вектори	n_clusters	Purity	ARI	NMI
ART-2 (Найкращий)	S-BERT	3	0.72	0.35	0.30
K-Means	S-BERT	3	0.90	0.74	0.61

В висновку K-Means показав удвічі вищий результат (ARI = 0.74), ніж найкраща конфігурація ART-2 (ARI = 0.35). Це є ключовим результатом всієї роботи.

Ключовим відкриттям стало те, що самі дані S-BERT є високоякісними, але алгоритм ART-2 не є оптимальним для роботи з ними.

1. Доказ якості даних: Еталонний тест з K-Means показав ARI = 0.74. Це означає, що S-BERT вектори утворюють 3 чіткі, геометрично розділені "хмари" у просторі, які K-Means легко знаходить.
2. Проблема ART-2: Найкращий результат ART-2 (ARI = 0.35) виявився вдвічі гіршим. Це свідчить про те, що ART-2, який є інкрементальною нейронною мережею і чутливий до порядку подачі даних та "прототипів", фундаментально гірше справляється з пошуком "центрів мас" у щільних, високорозмірних ембеддингах, ніж K-Means.

Висновки

У ході виконання лабораторної роботи було досліджено кластеризації текстових даних (arXiv) за допомогою мережі ART-2. Метою було засвоїти принципи роботи ART-1/ART-2 та отримати практичні навички застосування цих мереж для класифікації текстових векторів (TF-IDF і S-BERT). Також набув практичних навичок їх застосування для класифікації текстових даних, реалізувавши підготовку тексту, векторизацію (TF-IDF та S-BERT), нормалізацію, реалізація спрощеного ART-2, навчання з варіюванням параметрів, оцінка (Purity, NMI, ARI, Silhouette) та візуалізація

S-BERT значно покращує якість кластеризації в задачах, де важлива семантика (контекст слів). TF-IDF краще підходить для простих тематичних відмінностей і коли мало ресурсів. Для моїх експериментів ембеддинги зазвичай показували вищі NMI / Silhouette.

ρ – найважливіший гіперпараметр ART: при малих ρ мережа «зливає» теми (мало кластерів), при великих ρ – утворює багато дрібних кластерів. Для текстів у моїх експериментах оптимум лежав близько 0.85–0.89 (але це залежить від набору).

θ (пороговий поріг F_1) впливає на чутливість локальних активацій; η (learning rate) контролює швидкість адаптації прототипів; α – коефіцієнт зворотного зв'язку – впливає на домінування вихідного сигналу. Комбінація $\rho \approx 0.9$, $\theta = 0$, $\eta \approx 0.4$ –0.6 працювала стабільно.

Для завдань кластеризації тексту з використанням S-BERT ембедингів, рекомендується використовувати K-Means (якщо кількість кластерів відома) замість ART-2.